Day 2 - Mastering the Cursor Tool

Methodology and Requirements in SE

Tiago Sousa

What We Covered Yesterday

- **GenAI Fundamentals** - how large language models work and what they can/cannot do
- **VibeEngineering Introduction** - combining AI tools with engineering best practices
- **Tools Ecosystem** - the AI coding landscape and the autonomy spectrum
- **Installation and Configuration** - Cursor installed and configured
- **FreeSelfApp** - your first "naive prototype," built fast with AI

Same Tools, Different Discipline

Yesterday you explored. Today you engineer.

The tools did not change. Cursor is still Cursor.

What changes is how you use it.

Day 1 vs Day 2

	Day 1 (Yesterday)	Day 2 (Today)
Mindset	Explore, experiment, move fast	Plan, structure, build with discipline
Output	FreeSelfApp (naive prototype)	CalcApp v0.1 (structured, tested)
Approach	"Let the AI figure it out"	"Guide the AI with intent"
Tests	None (or very few)	TDD from the start
Planning	Minimal	Requirements-first, iterative plan
Tool depth	Surface level (install + first use)	Deep (agent mode, context, MCP, prompts)

Day 2 Agenda

Time	Session	Type	Duration
9:15	**Advanced Cursor Guided Demos**	Demo	90 min
10:45	*Break*		*15 min*
11:00	**Technical Planning Workshop**	Workshop	30 min
11:30	**CalcApp v0.1 - Part 1**	Workshop	45 min
12:15	*Lunch*		*60 min*
13:15	**CalcApp v0.1 - Part 2**	Workshop	45 min
14:00	**AI-Assisted Code Review - Intro**	Lecture	45 min
14:45	*Break*		*15 min*
15:00	**AI-Assisted Code Review - Practice**	Workshop	60 min
16:00	**Introduction to Git**	Lecture	45 min

The Artifact Chain

Today's sessions build on each other in a deliberate sequence:

1. **Cursor mastery** (9:15) - learn the tool deeply
2. **Technical planning** (11:00) - plan before you code
3. **CalcApp build** (11:30 + 13:15) - apply both skills with TDD
4. **Code review** (14:00 + 15:00) - review and document your code with AI
5. **Git** (16:00) - version and share your work properly

Each session feeds the next. By end of day: a working, tested, reviewed, versioned application.

Before We Start

- **Breaks** - 15 min at 10:45 and 14:45. Lunch is 60 min at 12:15.
- **Cursor readiness** - open Cursor now and make sure it launches.
- **CalcApp** - new project today (finance budgeting webapp).
- **Questions** - ask anytime.

Advanced Cursor Guided Demos

Session 02 - 90 min

What You'll Learn

By the end of this session, you will be able to:

1. **Structure a project** so Cursor's agent understands your codebase
2. **Write system prompts** using `.cursor/rules/*.mdc` files
3. **Manage context windows** with @-mentions and `.cursorignore`
4. **Use the planning agent** to break complex tasks into structured plans
5. **Extend Cursor with MCP** servers for external tools and data
6. **Activate Context7** for live, up-to-date library documentation

Why This Matters

> *If you hired a talented junior developer and dropped them into your project with no README, no coding standards, no architecture docs - how well would they perform?*

They might write good code, but it would be **inconsistent** with your patterns, in the **wrong directory**, using the **wrong conventions**.

The same is true for Cursor's agent.

Part 1

Project Structure

The "Before" - Unstructured Project

```
my-app/
├── app.ts
├── db.ts
├── helpers.ts
├── stuff.ts
├── test.ts
├── package.json
└── tsconfig.json
```

Ask the agent: "*Add a user registration endpoint with email validation.*"

Where does the route go? `app.ts`? Validation in `helpers.ts` or `stuff.ts`? Tests in `test.ts`?

The agent has to **guess**.

The "After" - Structured Project

```
my-app/
├── src/
│   ├── routes/
│   │   └── auth.routes.ts
│   ├── services/
│   │   └── auth.service.ts
│   ├── validators/
│   │   └── auth.validator.ts
│   ├── models/
│   │   └── user.model.ts
│   └── app.ts
├── tests/
│   ├── routes/
│   │   └── auth.routes.test.ts
│   └── services/
│       └── auth.service.test.ts
├── docs/
│   └── architecture.md
├── README.md
├── package.json
└── tsconfig.json
```

Same Prompt, Different Results

With structure, the agent:

- **Places** new code in the right directories
- **Follows** naming conventions it infers from existing files
- **Mirrors** the test structure automatically
- **Separates** concerns without being told

The project structure is your **first prompt** to the agent.

DEMO

Key Principles for AI-Friendly Structure

1. **Descriptive directory names** - `routes/`, `services/`, `models/`, `utils/` tell the agent what belongs where
2. **Mirror source structure in tests** - `src/services/auth.service.ts` maps to `tests/services/auth.service.test.ts`
3. **Include a README.md** - the agent reads it early and uses it to understand purpose, stack, and conventions
4. **Keep an architecture doc** - a simple `docs/architecture.md` helps the agent make structural decisions
5. **One module per file** - a 2000-line `app.ts` is hard for humans *and* AI

Part 2

System Prompts

How to set rules in Cursor

The approach is to use a `.cursor/rules/` directory with `.mdc` files (Markdown with Cursor metadata).

- **Multiple rule files**, each scoped to different parts of your project
- **Automatic activation** based on file patterns
- **Version controlled** with git

The `.mdc` File Format

```
---
description: "Enforce TypeScript strict mode and naming conventions"
alwaysApply: false
globs: ["**/*.ts", "**/*.tsx"]
---

# TypeScript Conventions

- Use `interface` over `type` for object shapes
- Prefer named exports over default exports
- Use PascalCase for types and interfaces
- Use camelCase for variables and functions
- All functions must have explicit return types
- No `any` types - use `unknown` if the type is truly unknown
```

YAML frontmatter controls *when* the rule activates. Markdown body is the instruction set.

The Four Rule Types

Type	Frontmatter	When It Activates
Always Apply	`alwaysApply: true`	Every conversation, every file
Auto-Attached	`alwaysApply: false` + `globs: [...]`	Files matching the glob pattern
Agent-Selected	`alwaysApply: false` + `description`	Agent decides based on description
Manual	No globs, no description	Only when you `@mention` it

Rule Hierarchy

When multiple rules apply, Cursor merges them with clear priority:

1. **Team Rules** (highest) - shared via repository settings
2. **Project Rules** - `.cursor/rules/*.mdc` committed to the repo
3. **User Rules** (lowest) - personal rules in Cursor settings

Project rules are the sweet spot: version-controlled, shared with the team, automatically applied.

A Real-World Rules Setup

```
.cursor/
└── rules/
    ├── base.mdc            # Always Apply - project overview, tech stack
    ├── typescript.mdc       # Auto-Attached - TS conventions (globs: **/*.ts)
    ├── react.mdc           # Auto-Attached - React patterns (globs: **/*.tsx)
    ├── testing.mdc         # Auto-Attached - test conventions (globs: **/*.test.ts)
    ├── api.mdc             # Auto-Attached - API patterns (globs: src/routes/**)
    └── database.mdc        # Agent-Selected - DB schema and migration rules
```

The `base.mdc` file is the most important - it is the first thing the agent reads.

Example: `base.mdc`

```
---
description: "Core project context and conventions"
alwaysApply: true
---

# CalcApp - Finance Budgeting Application

## Tech Stack
- Runtime: Node.js 20
- Language: TypeScript 5.x (strict mode)
- Framework: Express.js
- Database: PostgreSQL with Prisma ORM
- Testing: Vitest + Supertest
- Package manager: npm

## Architecture
- Layered: Routes -> Services -> Repositories
- Business logic in services, never in routes
- Routes handle HTTP concerns only

## Conventions
- All code and comments in English
- Conventional Commits for commit messages
- Every new feature must include tests
```

Part 3

Context Windows & @-Mentions

What Is a Context Window?

Every interaction with Cursor's agent assembles a **context window** - the total information sent to the AI model.

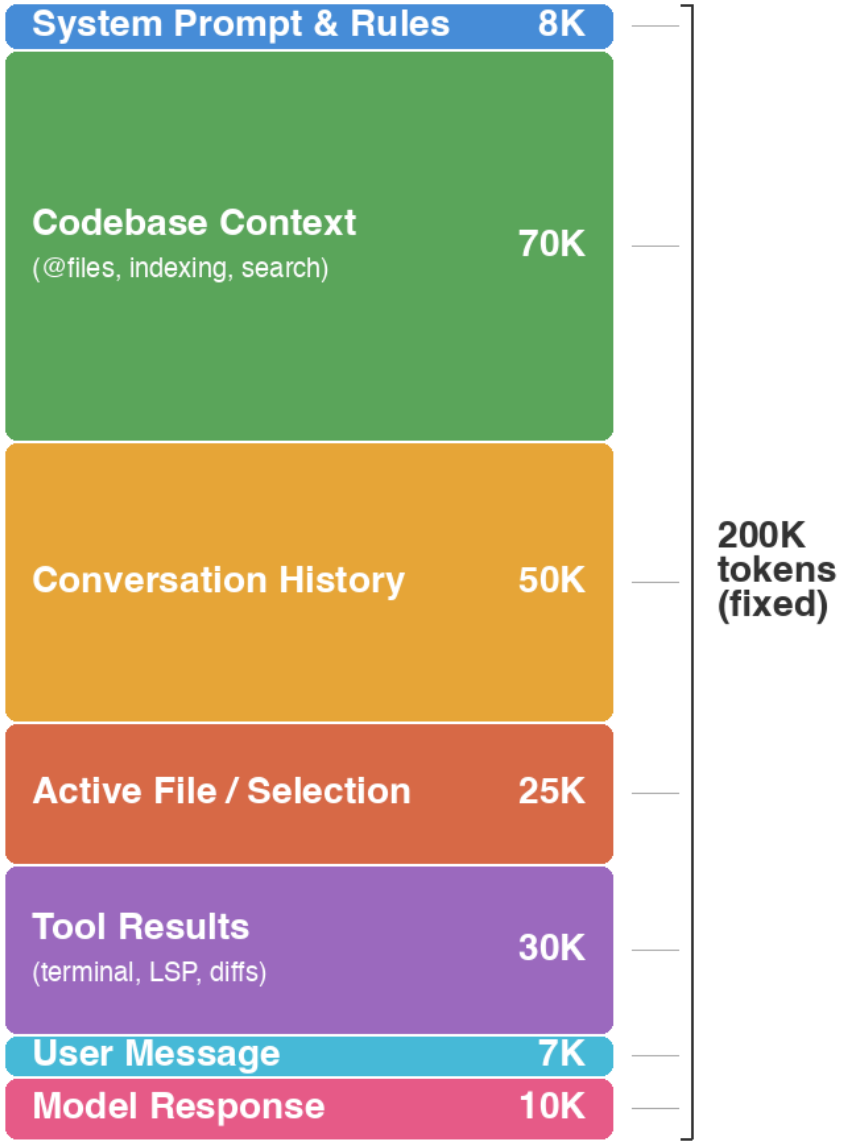
Everything must fit inside this fixed-size window:

- Your prompt (what you typed)
- System prompts (your `.mdc` rules)
- Referenced files
- Conversation history
- The agent's working memory

When it fills up, older information gets dropped. The agent can "forget" earlier instructions.

Context Window Composition

Cursor · 200K tokens



Key Insight

- The context window is finite and shared.
- Every section competes for the same 200K tokens.
- Large codebase context = less conversation history.
- When the window fills up, oldest messages are dropped.
- Strategic use of @-mentions beats indexing everything.

When the Window Is Full...

- ☐ Older conversation turns get truncated
- ☐ The model 'forgets' earlier instructions
- ☐ Code context may be silently dropped
- ☐ Responses lose coherence with prior work

Best Practice

Keep context focused. Use @-mentions for specific files.
Start new conversations when context gets stale.

@-Mentions: Precision Context

Instead of hoping the agent finds the right files, point it explicitly:

@-Mention	What It Does	When to Use
`@Files`	Adds a specific file	You know which file is relevant
`@Folders`	Adds directory structure	Agent needs a module's layout
`@Code`	Adds a function or class	Reference a particular symbol
`@Docs`	Searches indexed docs	Need framework/library info

@-Mentions in Practice

Vague prompt (bad):

```
Add password reset to the project.
```

Precise prompt with @-mentions (good):

```
Looking at @auth.service.ts and @user.model.ts,
add a password reset flow that follows the same
patterns used in the registration flow.
```

You give the agent exactly the context it needs - nothing more, nothing less.

`.cursorignore`: Keeping Noise Out

Works like `.gitignore` but for Cursor's indexing:

```
# .cursorignore

# Dependencies - massive, never needed in context
node_modules/

# Build output - generated, not source
dist/
build/

# Sensitive files
.env
.env.local
```

Without this, Cursor may waste context indexing hundreds of thousands of files in `node_modules/`.

Context Management Tips

1. **Start fresh conversations for new tasks** - don't let history eat your context window
2. **Be specific with @-mentions** - `@auth.service.ts` beats letting the agent search your entire codebase
3. **Keep files focused** - a 500-line file costs less context than a 3000-line file
4. **Watch for degradation** - if the agent starts ignoring rules mid-conversation, start a new chat

Checkpoint 1

Hands-on: Create your first system prompt

1. Open Cursor with your CalcApp project (or a fresh project)
2. Create `.cursor/rules/base.mdc`
3. Add `alwaysApply: true` in the frontmatter
4. Include your tech stack and at least 3 coding conventions
5. Ask the agent: "*Create a utility function that formats currency values*"

Observe: Does the agent follow your conventions?

Experiment: Remove `alwaysApply: true` and retry. What changes?

Part 4

Planning Agent

The Problem with "Just Do It"

Simple tasks work fine in Act mode.

But what about:

> *"Add user authentication with JWT tokens, including login, registration, token refresh, and role-based access control."*

The agent starts coding immediately. It might nail login but forget token refresh. Or implement everything in the wrong structure.

The issue is not capability - it's diving into execution before thinking.

Plan Mode vs. Act Mode

Toggle with **Shift+Tab**

Plan Mode - analyzes and produces a structured plan *without writing code*:

- What files to create or modify
- The order of changes
- Key decisions and trade-offs
- A checklist of subtasks

Act Mode - executes code changes, following the plan step by step

DEMO

The Plan-Then-Act Workflow

1. **Switch to Plan mode** (Shift+Tab)
2. **Describe the full feature** - be specific about requirements and constraints
3. **Review the plan** - check coverage, file locations, architecture
4. **Adjust** - "move validation into a separate service" or "add error handling"
5. **Switch to Act mode** (Shift+Tab) and let it execute

This mirrors how experienced developers work: **think first, then code.**

When to Use Plan Mode

Scenario	Plan Mode?	Why
Fix a typo in a string	No	Trivial, one file
Add a new API endpoint	Maybe	Depends on complexity
Implement authentication	**Yes**	Multi-file, architectural decisions
Refactor module structure	**Yes**	High impact, needs ordering
Add a CSS style	No	Simple, isolated
Build a feature with tests	**Yes**	Multiple files, test coverage

Rule of Thumb

If the task requires decisions about *where* code goes or *how* components interact, plan first.

The Todo System

When the agent works in Plan mode, it creates an internal checklist (the "todo" system):

- Tracks progress through each subtask
- Checks items off as it completes them in Act mode
- Visible in the conversation

This lets you:

- See where the agent is in execution
- Intervene if it goes off track
- Verify completeness before moving on

Part 5

Model Context Protocol (MCP)

What Is MCP?

The agent needs to interact with things *outside* your codebase:

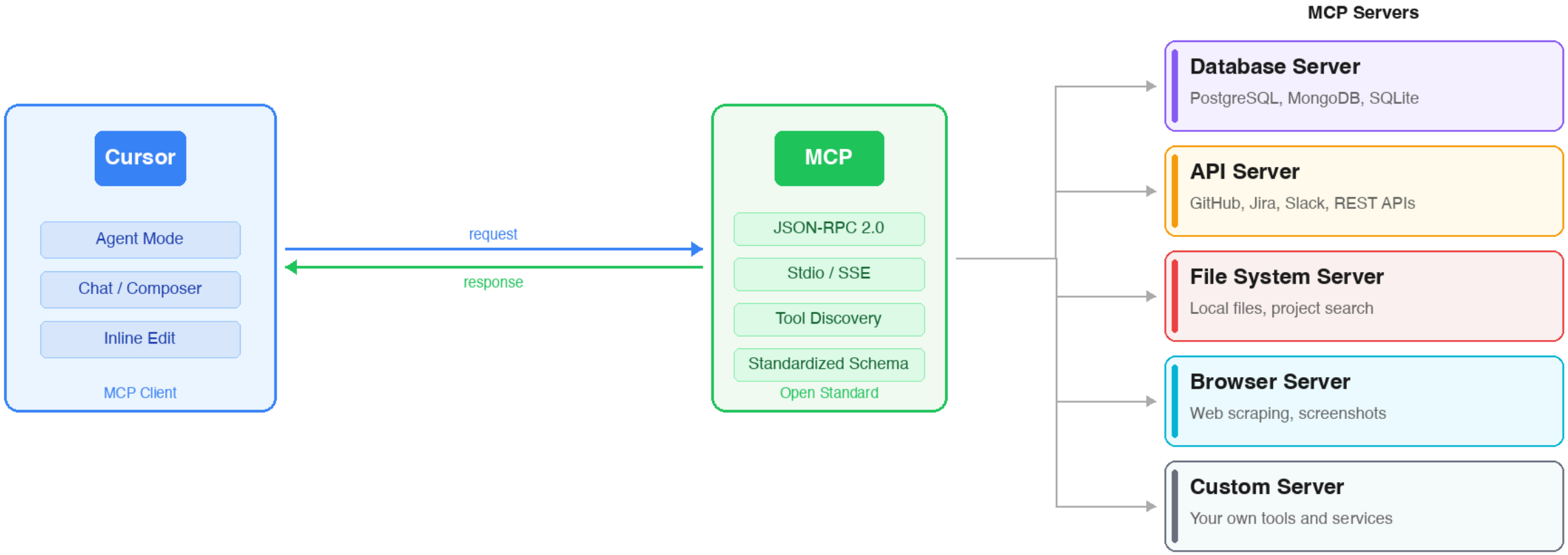
- Query a database
- Check deployment status
- Look up library documentation

MCP (Model Context Protocol) is a standard that lets Cursor connect to external services.

Think of it as a **plugin system**: each MCP server exposes "tools" the agent can call during a conversation.

Model Context Protocol (MCP)

A standard interface connecting AI coding tools to external services



One protocol, any service - MCP servers are interchangeable and reusable across tools

@ Right-On-Skill

Project-Level MCP Config

Committed to git, shared with the whole team:

```
// .cursor/mcp.json
{
  "mcpServers": {
    "context7": {
      "url": "https://mcp.context7.com/mcp"
    },
    "database": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "DATABASE_URL": "postgresql://localhost:5432/calcapp"
      }
    }
  }
}
```

Global MCP Config

Personal tools, not shared with the team:

```
// ~/.cursor/mcp.json
{
  "mcpServers": {
    "my-private-tool": {
      "command": "node",
      "args": ["/home/you/tools/my-mcp-server.js"]
    }
  }
}
```

Limit: Cursor supports a maximum of **40 tools** combined across all MCP servers. Keep your config focused.

Project vs. Global MCP

Level	File Location	Use Case
Project	`.cursor/mcp.json`	Team tools: shared DB, project docs, CI/CD
Global	`~/.cursor/mcp.json`	Personal tools: private APIs, local utilities

Best practice: commit `.cursor/mcp.json` to your repo. Every team member gets the same tools automatically.

Use environment variables for sensitive values (database passwords, API keys).

How the Agent Uses MCP

Once configured, MCP tools appear **automatically** in the agent's capabilities.

You don't need to invoke them explicitly. When the agent needs external data, it calls the right tool.

Agent calling a MCP

> *"What tables exist in our database?"*

The agent calls `list_tables`, gets the result, and shows you - all within the conversation.

DEMO - MCP Server

Part 6

Context7

The Stale Documentation Problem

You ask Cursor to help with a library. It generates code based on an **outdated API**.

The model's training data has a cutoff date. It may not know the latest version of React, Express, Prisma, or any library you use.

Context7 pulls **live, up-to-date documentation** directly into your conversations.

Setting Up Context7

One line in your `.cursor/mcp.json`:

```
{
  "mcpServers": {
    "context7": {
      "url": "https://mcp.context7.com/mcp"
    }
  }
}
```

No API key. No local installation. No configuration.

The server runs remotely and covers hundreds of popular libraries.

Using Context7

Add **"use context7"** to your prompt:

```
Using Express.js, create a middleware that validates JWT tokens.
Use context7 for the latest Express and jsonwebtoken API.
```

The agent fetches current docs, then generates code matching the **latest** API.

DEMO

Without vs. With Context7

	Without Context7	With Context7
Source	Model training data (months old)	Live documentation
Risk	Deprecated methods, old patterns	Current API
Debugging	Time spent on API mismatches	Minimal
When to use	Never (if Context7 is available)	Always with fast-moving libraries

Use Context7 whenever you work with libraries that release frequently: React, Next.js, Prisma, Express, etc.

Checkpoint 2

Hands-on: Use the planning agent

1. Use this prompt:

> *"Plan the implementation of a monthly expense tracking feature. Users should be able to add expenses with a category, amount, and date. Include a service layer, route handlers, and tests."*

2. Review: Does the plan follow your `base.mdc` architecture? Right directories? Tests included?

Key Takeaways

What to Remember

1. **Project structure is your first prompt** - invest in organization; it pays off on every interaction
2. `.cursor/rules/*.mdc` **replaces** `.cursorrules` - four activation types, clear hierarchy, version-controlled
3. **Context windows are finite** - use @-mentions for precision, `.cursorignore` for noise, fresh conversations for clarity
4. **Plan before you act** - Shift+Tab for any task touching 2+ files. Review the plan, then execute.
5. **MCP extends Cursor beyond code** - connect to databases, APIs, docs via `.cursor/mcp.json`

Break

15 minutes - back at 11:00

Workshop: Technical Planning

From Vision to Actionable Plan

Learning Objectives

- Translate a high-level product vision into structured requirements using Cursor agent as a brainstorming partner
- Decompose user stories into atomic, traceable todos that an AI agent can execute one by one
- Apply the planning agent workflow to a real project from scratch

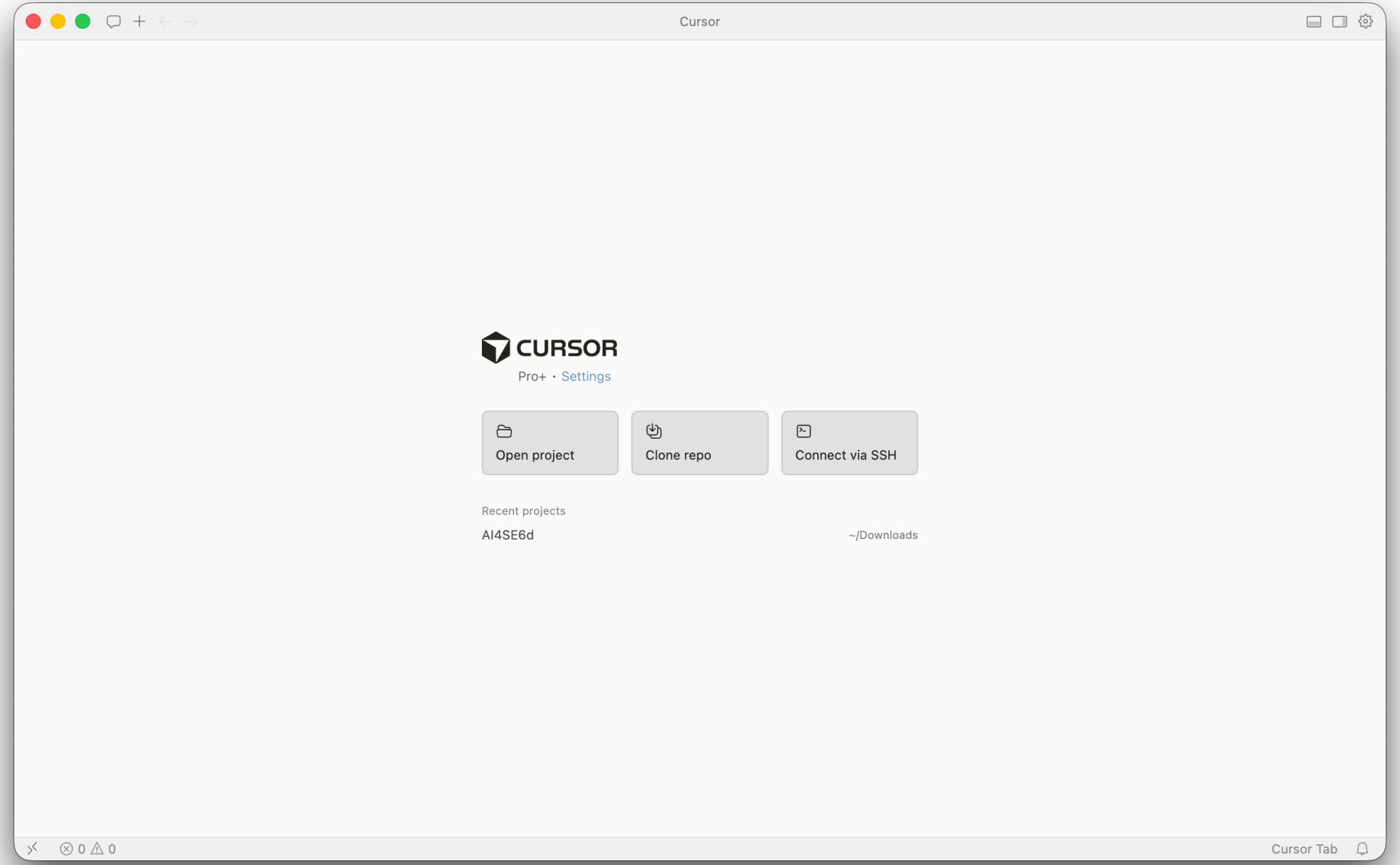
The Blank-Page Problem

You know how Cursor works. But knowing the tool and knowing **how to start a project** are different skills.

The hardest moment: empty repo, blank editor, no direction.

This is **blank-page syndrome** - and it hits harder with AI because the agent needs clear direction before it can help.

This workshop gives you a structured way to start.


Cursor
CURSOR
Pro+ · Settings
Open project
Clone repo
Connect via SSH
Recent projects
AI4SE6d
~/Downloads
0 0
Cursor Tab

Introducing CalcApp

Personal finance budgeting web application

Track monthly expenses, set budgets per category, see how much you have left to spend.

A focused alternative to complex finance apps - just the features that matter for day-to-day budget awareness.

CalcApp Core Features

Feature	Description
Add expenses	Amount, category (groceries, transport, etc.), date
View expenses	List filtered by month and category
Set monthly budgets	Per category (e.g., max 300 EUR on groceries)
See remaining budget	Visual indicators when close to or exceeding limits

Tech stack: TypeScript frontend + backend, built with TDD from the start.

Minimum Feature Set

Your CalcApp **must** include these four capabilities:

1. Add expense
2. List expenses
3. Set budget per category
4. Show remaining budget

Beyond this minimum, you are free to add your own ideas.

Take 2 minutes now: What other features would make this app useful for you? Jot down 2-3 ideas.

Requirements Elicitation with Cursor

10 minutes

The Starter Prompt

Write this into Cursor agent mode - adapt it with your own feature ideas:

```
I'm building CalcApp, a personal finance budgeting web app in TypeScript.

Core features:
- Add expenses (amount, category, date)
- List expenses filtered by month and category
- Set monthly budgets per category
- Show remaining budget per category with visual indicators

Help me derive a complete set of user stories for an MVP. For each story, include:
A clear title
2. A one-sentence description ("As a user, I want to...")
3. Acceptance criteria (testable conditions that define "done")

Start with the four core features, then suggest 2-3 additional stories that would make the app more useful.
```

What to Do

1. **Open Cursor** - new agent chat session
2. **Paste the starter prompt** (modified with your ideas)
3. **Read critically** - don't just accept. Are stories clear? Criteria testable? Anything missing?
4. **Iterate** - ask follow-ups:
    - "What about error handling?"
    - "Should we include data persistence?"
    - "Fixed categories or user-defined?"
5. **Converge on 6-10 user stories** for your MVP

What Good Requirements Look Like

Specific enough to test

> *"When I add an expense with amount 0 or negative, the app shows a validation error"*
> *NOT: "The app should handle expenses"*

Small enough for one session

> *If a story takes more than 30-45 min of AI-assisted coding, break it down*

Independent enough to build in any order

> *Stories that require other stories first create bottlenecks*

Decomposition: Stories to Todos

9 minutes

Why Atomic Todos Matter

An AI agent works best when each instruction is:

- **One thing** - "Create the Expense model with amount, category, and date fields"
- **Verifiable** - produces a result you can check (file created, test passing, component rendering)
- **Ordered logically** - foundation first, then features, then refinements

Think of it like giving directions in a new city:

- "Go to the library" - unhelpful
- "Walk north on Main Street for two blocks, turn left on Oak Avenue, third building on the right" - gets them there

The Decomposition Exercise

Pick your **two most important** user stories. Prompt Cursor:

```
Take this user story and break it into atomic
implementation todos. Each todo should be a single,
verifiable step. Order them so each step builds on
the previous one. Include test-writing steps
(TDD: write the test before the implementation).

User story: [paste your story here]
```

Example: Good Atomic Todos

1. Create `Expense` interface with `amount: number`, `category: string`, `date: Date`
2. Write unit test: creating an Expense with valid data succeeds
3. Write unit test: creating an Expense with negative amount throws validation error
4. Implement Expense validation logic to pass both tests
5. Write unit test: `addExpense()` stores an expense in the list
6. Implement `addExpense()` function to pass the test

Notice the pattern: **test first, implement second**.

This is TDD - and it is how you will build CalcApp throughout the course.

DEMO

Share and Review

1. **Your user stories** - what did the agent produce? What did you change?
2. **Your atomic todos** - how granular? Did the agent produce good TDD-ordered steps?

Different prompts lead to different plans. That is not a bug - it is the nature of requirements work.

The goal: plans that are **clear, testable, and actionable**.

Checkpoint

Before moving on, verify you have:

- **6-10 user stories** with title, description, and acceptance criteria
- **Atomic todos** for at least your two most important stories (TDD-ordered)
- **A clear starting point** - you know which todo to tackle first

Missing something? Use remaining time to fill the gaps.

This plan is your roadmap for the afternoon coding sessions.

Session 03: Key Takeaways

1. **Start with a vision, not with code** - turn a vague idea into clear, testable requirements first
2. **A good starter prompt defeats blank-page syndrome** - structured prompt with context, features, and expected output
3. **Atomic todos bridge planning and coding** - user stories say *what*, todos tell the AI *how*
4. **TDD order matters** - test before implementation in every todo
5. **Requirements elicitation is a skill, not a formality** - plan quality determines code quality

CalcApp v0.1 - Part 1

Test-Driven Development with AI

Learning Objectives

- Apply the **Red-Green-Refactor** cycle with AI assistance
- Scaffold a project from scratch using Cursor agent
- Practice **AI-TDD discipline** - human writes tests, AI implements
- Deliver **two passing features** before lunch

Why Start with Tests?

The #1 risk with AI-assisted coding: **losing control**.

When the AI generates code freely:

- Output comes fast
- But you have no idea if it actually works
- No way to verify beyond manually clicking through the app

Tests are your safety net. They tell you in milliseconds whether code does what you intended.

The core VibeEngineering discipline:

> ***You set the specification (the test). The AI does the implementation. You stay in the driver's seat.***

TDD in 5 Minutes

Show, Then Explain

Instead of explaining TDD theory first, we will **watch a complete cycle** and explain afterward.

Code first. Theory later.

Instructor Demo: A Complete Cycle

Step 1 - Ask AI to write a test

```
Implement a test for a React component called `ExpenseItem` that takes an `expense` prop
 (with fields: id, amount, description, category, date, createdAt) and an `onDelete` handler.

When rendered, the component should display:
- The description of the expense.
- The amount, formatted as euros (e.g., "€25.50" for an amount of 25.50).

Example: If given an expense with description "Lunch" and amount 25.50,
the component output should include both "Lunch" and "€25.50" in the document.
```

Demo: Red, Green, Refactor

Step 2 - Run the test: it fails (RED). Component doesn't exist yet.

Step 3 - Prompt Cursor:

> *"Implement the minimal ExpenseItem component to pass this test."*

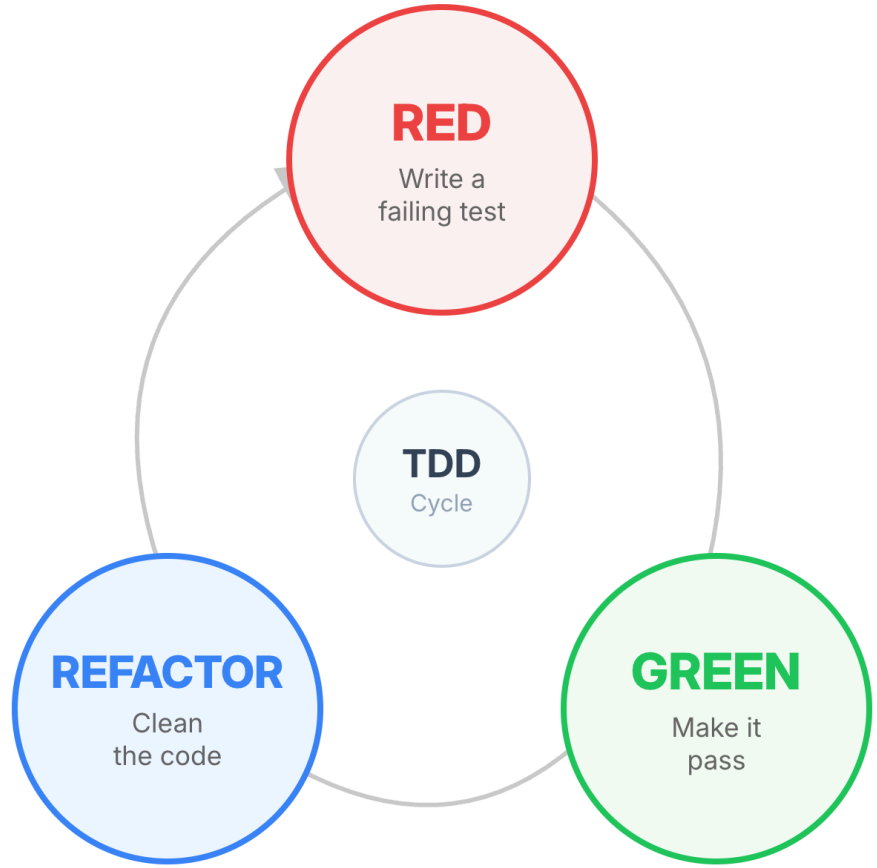
Step 4 - Watch Cursor generate the implementation. Review it.

Step 5 - Run the test again: it passes (GREEN).

Step 6 - Refactor if needed. Clean up without breaking the test.

TDD Cycle: Red - Green - Refactor

Write a failing test, make it pass, then clean up



Never write production code without a failing test first

The AI-TDD Discipline

Step	Who	What
Red (write the test)	**You + AI**	Ask Cursor to write the failing test. You review it - this is your specification.
Green (write the code)	**The AI** (Cursor agent)	Prompt: "Implement minimal code to pass this test."
Refactor (clean up)	**You + AI together**	You review and decide what to improve.

The Golden Rule

Never let the AI write both the test and the implementation without YOUR supervision.

If the AI writes the test, it may write a test that matches whatever it was going to implement anyway - that proves nothing.

AI-TDD Workflow

Human writes the test, AI writes the code, together you refine

1

RED

Write the failing test

YOU (Human)

This is your specification.
You define the behavior.

→

2

GREEN

Make it pass

AI (Cursor Agent)

Prompt: "Implement minimal
code to pass this test."

→

3

REFACTOR

Clean the code

YOU + AI Together

You review and decide
what to improve.

The Golden Rule

Never let the AI write both the test and the implementation without YOUR supervision.
If the AI writes the test, it may write a test that matches whatever it was going to implement anyway - that proves nothing.

The Mutant Test Trick

After reaching GREEN (test passes):

1. **Intentionally break** the implementation
2. **Run the test** - does it fail?
3. **Yes?** Your test is real. Undo the change.
4. **No?** Your test isn't testing anything useful. Fix it.

This takes **10 seconds** and saves hours of false confidence.

Example: Change `.toFixed(2)` to `.toFixed(0)` in ExpenseItem. If the test still passes, the test is lying to you.

Scaffold from Scratch

10 minutes

The Scaffold Prompt

Open Cursor in an **empty directory**. Use this exact prompt in agent mode:

```
Create a new TypeScript web application called "CalcApp" –
a personal finance budgeting tool. Use this exact stack:
- Vite + React + TypeScript
- Vitest for unit testing with jsdom environment
- react-router (not react-router-dom) for page navigation
- Plain CSS for styling (no external UI library)

Create all files directly. Do NOT use create-react-app or
any interactive CLI tool. Include Vitest test config inside
vite.config.ts, one passing example test, and a minimal App
component that renders "CalcApp". Put tests in a tests/
directory (not co-located in src/). After creating files,
run npm install then npm test.
```

The Canonical Tech Stack

Layer	Technology
Build tool	Vite
UI framework	React
Language	TypeScript
Test runner	Vitest
Test utilities	React Testing Library + jsdom
Routing	react-router
Styling	CSS (plain)

No Jest. No Tailwind. No Material UI. No external component libraries.

Plain CSS keeps styling simple and self-contained.

Expected Project Structure

After scaffolding, you should have, something like:

```
CalcApp/
  src/
    App.tsx
    App.css
    main.tsx
    components/
      ExpenseForm.tsx
      xyz.tsx
    pages/
      Dashboard.tsx
      AddExpense.tsx
  tests/
    setup.ts
    components/
  index.html
  package.json
  vite.config.ts
  tsconfig.json
```

Verify it works:

```
npm test          # Should see 1 passing test
npm run dev       # Should show "CalcApp" in browser
```

DEMO

Scaffold Rescue Kit

Problem	Symptom	Fix
Wrong framework	Next.js or plain HTML generated	Re-prompt: "Vite + React, NOT Next.js"
Test framework broken	Vitest fails to run or find tests	Use the rescue config (next slide)
npm install fails	Dependency conflicts	Raise your hand - instructor will help
Cursor hangs	Waiting for interactive input	Ctrl+C, re-prompt with "Do NOT use any interactive CLI tool"

Vitest Rescue Config

If your test setup is broken, replace `vite.config.ts` with:

```
/// <reference types="vitest/config" />
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: ['./tests/setup.ts'],
    css: true,
  },
})
```

First Feature: ExpenseItem

Red - Green - Refactor

Step 1: Configure Cursor Rules

Before writing code, create `.cursor/rules` in your project root:

```
Testing rules:
- ALWAYS write the test file BEFORE the implementation file
- Use Vitest with React Testing Library
- Put test files in the tests/ directory (NOT co-located in src/)
- Use fireEvent from @testing-library/react (NOT userEvent)
- Use react-router (NOT react-router-dom) for imports
- Test behavior, not implementation details
Do NOT:
- Install or suggest Jest (we use Vitest)
- Skip writing tests
```

These rules steer Cursor away from bad habits (suggesting Jest, skipping tests, wrong imports).

Step 2: Write the Failing Test (RED)

Ask Cursor to write this test. **You** review it before moving on.

> *Generate a test that checks that the `ExpenseItem` component displays the expense's amount (formatted as `€25.50`) and its description ("Lunch"). When you ask Cursor to generate the component, your goal is to get a minimal `ExpenseItem` that receives an expense and shows both the formatted amount and description as shown in the test. Write in tests/components/ExpenseItem.test.tsx*

Step 3: Prompt Cursor (GREEN)

Prompt Cursor agent:

> *"Implement the minimal ExpenseItem component to pass the existing test in tests/components/ExpenseItem.test.tsx. Do not modify the test file. Use a default export."*

Expected result, should pass. Test with:

```
npx vitest run
```

Step 4: Mutant Test + Refactor

Mutant test: Change `.toFixed(2)` to `.toFixed(0)`. Run tests. Does it fail? Good - your test catches real bugs. Undo the change.

Refactor ideas:

- Show category and date alongside amount and description?
- Split "displays amount and description" into two separate tests?

Re-run tests after any refactoring to confirm they still pass.

Second Feature: ExpenseForm

Your turn - less guidance this time

Step 1: Ask Cursor to Write the Failing Test

Prompt Cursor:

> *"Write a Vitest test for an ExpenseForm component in tests/components/ExpenseForm.test.tsx. The test should render the form, fill in amount (4.50), description (Coffee), category (Food), and date (2026-03-01) using labeled inputs, click an 'Add Expense' button, and assert that onSubmit was called with the correct expense data."*

Review the generated test. Check that it:

- Imports from the right paths
- Tests the behavior you actually want
- Uses accessible queries (`getByLabelText`, `getByRole`)

Then run it:

```
npx vitest run   # Should FAIL — ExpenseForm doesn't exist
```

Step 2: Prompt Cursor (GREEN)

> *"Implement the minimal ExpenseForm component to pass the existing test in tests/components/ExpenseForm.test.tsx. Use a controlled form with labeled inputs for amount, description, category, and date. Use a default export. Do not modify the test file."*

Review the generated code. Run the test:

```
npx vitest run   # Should PASS
```

Step 3: Mutant Test + Refactor

Break something small: remove the `parseFloat` on the amount. Does the test fail? Good. Fix it. Refactor if needed.

You now have two tested features:

- Displaying an expense item
- Submitting an expense

Pre-Lunch Checkpoint

Run these three checks:

```
# 1. All tests pass
npx vitest run

# 2. Dev server starts
npm run dev
# Open the URL - you should see "CalcApp"

# 3. Check test count
npx vitest run --reporter=verbose
# At least 2 passing tests (3 counting scaffold example)
```

Minimum Viable Outcome

You need:

- 2+ passing tests (ExpenseItem + ExpenseForm)
- App renders in the browser
- Dev server runs without errors

Behind? Don't worry. Nobody gets left behind.

Ahead? Write a third test for a feature from your plan:

- Display a list of expenses
- Calculate the total

Follow the same Red-Green-Refactor cycle.

Session 04: Key Takeaways

1. **Test first, implement second** - Red-Green-Refactor keeps you in control with AI
2. **Never let the AI write both test and implementation blindly** - the cardinal rule of AI-TDD
3. **Scaffold with explicit instructions** - specify exact stack, forbid interactive CLI tools
4. **Use the mutant test** - break the code, verify the test catches it (10 seconds)

CalcApp v0.1 - Part 2

Routing, Data Layer & UI Polish

Learning Objectives

After this session, you will be able to:

1. **Add page routing** to a React app using React Router, driven by tests
2. **Implement a local data layer** with React state shared across pages
3. **Apply minimal UI polish** for a functional, readable interface
4. **Complete v0.1** with 4+ passing tests covering routing, data, and UI

Post-Lunch Re-orientation

Before we start: **verify your work from Part 1 still works.**

Re-orientation Checklist

1. Open a terminal in your CalcApp project directory
2. Run `npm test` - verify tests from Part 1 pass
3. Run `npm run dev` - verify the app renders in browser
4. If either fails: use the checkpoint files (next slide)
5. Open Cursor and review your plan from Session 03

Catch-up: Part 1 Checkpoint

If your project is broken, ask the instructor for the checkpoint files.

You will get: 2 passing tests, basic components, Vite + React + TypeScript + Vitest.

Section 1: Routing & Navigation

Two Pages for CalcApp

Route	Page	Purpose
`/`	Dashboard	Shows all recorded expenses with totals
`/add`	Add Expense	Form to enter a new expense

React Router gives you client-side navigation between these views without full page reloads.

Step 1: Ask Cursor for a Routing Test

Remember: **ask Cursor to write the test, you review it, then Cursor implements the code.**

Prompt Cursor:

> *"Write a Vitest test in tests/App.test.tsx that verifies the App component renders navigation links for 'Dashboard' and 'Add Expense'. Wrap App in a MemoryRouter for testing. Use screen.getByRole('link') to find the links. The App component does not have routing yet - this test should fail."*

Review the generated test. Then run it:

```
npx vitest run   # Should FAIL (red) — routing not implemented yet
```

Step 2: Prompt Cursor to Implement Routing

```
Install react-router (NOT react-router-dom) if not already
installed. Use createBrowserRouter and RouterProvider for
routing with two routes:
- "/" renders a Dashboard page
- "/add" renders an AddExpense page

Create a Layout component that uses Outlet from react-router.
The Layout has a nav bar with "CalcApp" as h1 and Link
components for "Dashboard" and "Add Expense".
Create placeholder page components that render their
page name as a heading.
Make the existing routing test pass.
```

Run `npm test` - your routing test should now pass (green).

Review What Cursor Produced

Check these four things:

- `react-router` (not `react-router-dom`) added to `package.json`?
- App uses `createBrowserRouter` and `RouterProvider`?
- Navigation uses `Link` components (not `<a>` tags)?
- Each page component in its own file under `src/pages/`?

Always review AI-generated code. Trust but verify.

Step 3: Route-Specific Test

Ask Cursor to add a second test:

> *"Add a test to the same file that navigates to '/add' using MemoryRouter's initialEntries and verifies that a heading with 'Add Expense' is rendered."*

Review it, then run. This should already pass if Cursor implemented routes correctly.

> ***Note:*** `MemoryRouter` *imports from* `react-router` *(not* `react-router-dom`*). This is the correct import for the latest React Router.*

Section 2: Data Layer

The Data Model

The core interface in `src/types/expense.ts`:

```
interface Expense {
  id: string;
  amount: number;
  description: string;
  category: string;
  date: string;
  currency?: string;       // optional, later iterations
  recurring?: boolean;      // optional, later iterations
  createdAt: string;        // ISO timestamp
}
```

Data persists in localStorage via storage utilities in `src/utils/storage.ts` (`getExpenses`, `saveExpenses`). No backend needed for v0.1.

Ask Cursor to Write a Data Flow Test

This test verifies the full cycle: add an expense, then see it in the list.

Prompt Cursor:

> *"Write a Vitest integration test that starts on the '/add' route, fills in an expense form (amount: 42.50, category: Food, description: Lunch) using fireEvent, clicks the Add button, then navigates to Dashboard and asserts that '42.50' and 'Food' appear on the page. Use MemoryRouter with initialEntries. The data layer does not exist yet - this test should fail."*

Review the generated test. Run it:

```
npx vitest run   # Should FAIL (red) — data layer not implemented yet
```

Prompt Cursor for the Data Layer

```
Implement the data layer for CalcApp:
1. Define the Expense type in src/types/expense.ts
2. Create storage utilities in src/utils/storage.ts
   with getExpenses() and saveExpenses() using
   localStorage (key: "calcapp-data")
3. Dashboard page (at /) reads expenses from storage
   and displays them using ExpenseList
4. AddExpense page (at /add) saves new expense to
   storage and navigates to "/" using useNavigate
   from react-router
5. Make the failing test pass.
```

Run `npm test` - green.

Review Checklist for Data Layer

- Types defined cleanly in `src/types/expense.ts`?
- Storage utilities in `src/utils/storage.ts` with `getExpenses` / `saveExpenses`?
- Form uses controlled components (value + onChange)?
- AddExpense navigates back to `/` after submission using `useNavigate`?

Section 3: Minimal UI Polish

What "Done" Looks Like for v0.1

- Visible navigation bar at the top of every page (Dashboard + Add Expense)
- Dashboard displays expenses as a formatted list with totals
- Add expense form with labeled fields and spacing
- Text readable, nothing overlapping, layout works

Target: "readable and functional," not "production-ready."

Prompt Cursor for Styling

```
Add minimal CSS styling using plain CSS (App.css):
1. Horizontal nav bar with links as tabs/buttons
2. Dashboard: expense list showing description,
   category, amount, date, with totals
3. AddExpense form: labels, input spacing, styled
   submit button
4. Simple color scheme. No animations, no shadows,
   no external CSS libraries.
5. Centered layout with reasonable max-width.
```

> *Do not spend more than 10 minutes on styling.*

v0.1 Checkpoint

v0.1 Completion Checklist

- **Routing works** - two pages accessible via nav links
- **Add expense** - form submits, data persists in localStorage
- **List expenses** - all added expenses appear on Dashboard
- **Storage persistence** - data survives page refresh
- **4+ tests passing** - `npm test` shows green
- **Dev server runs** - `npm run dev` shows working app
- **Clean code** - no TypeScript errors, no console warnings

What You Just Built (in 90 Minutes)

- Fully scaffolded TypeScript + React project with testing
- Test-driven features: **you** wrote specs, **AI** wrote code
- Client-side routing with two functional pages
- Working data flow: form input to localStorage to display
- Basic but readable styling

This is VibeEngineering: you controlled the design decisions, Cursor handled the implementation.

If You Did Not Finish

Ask for help. You need a working CalcApp for the code review sessions this afternoon.

Verify everything works:

```
npm install
npm test
npm run dev
```

Key Takeaways

1. **TDD works with AI** - ask Cursor to write the test first, review it, then let Cursor implement. You define "correct" by reviewing, AI writes the code.
2. **Routing, data, UI are separate concerns** - tackle them in sequence, not all at once.
3. **localStorage is enough for v0.1** - no Redux, no backend needed for a working prototype.
4. **"Done" = tests pass**, not pixels are perfect.
5. **Checkpoint files are safety nets**, not failures.

AI-Assisted Code Review & Living Documentation

Is the code good? How do you know?

Learning Objectives

After this session, you will be able to:

1. **Explain why code review is more critical** with AI-generated code
2. **Compare traditional and AI-assisted review** approaches
3. **Use Cursor as a code reviewer** with effective prompts
4. **Define living documentation** and contrast it with static docs
5. **Apply the 80/20 rule** for AI-assisted documentation

The Uncomfortable Question

You spent the morning building CalcApp.

You wrote code. Cursor helped you write code.

At the end, you had something that works.

But is the code good? How do you know?

The Speed Paradox

When you wrote code by hand, you had a natural throttle on bugs.

You could only type so fast.

AI removes that throttle entirely.

> *AI writes code faster, but it also writes bugs faster.*
> *Code review is **MORE** important with AI, not less.*

The Numbers: AI-Generated Code Quality

Metric	Human-Written	AI-Generated	Difference
Issues per PR	6.45	10.83	**1.7x more**
XSS vulnerabilities	baseline	2.74x higher	**2.74x more**

Source: CodeRabbit 2025 study, thousands of pull requests analyzed.

60 Million AI-Assisted Code Reviews

Since April 2025, GitHub reports over **60 million** Copilot-assisted code reviews.

That number tells you something profound:

- The volume of AI-generated code is **enormous**
- Someone (or something) needs to check **all of it**
- Speed without review is just **faster failure**

The Single Most Effective Quality Practice

Capers Jones analyzed **12,000+ software projects**:

Formal code review catches 60-65% of defects

No other practice comes close on its own.
Not unit testing. Not integration testing. Not QA.

Code Review Is More Than Bug Catching

- **Knowledge sharing** - reviewers learn how other parts work. Bus factor drops.
- **Consistency** - teams catch deviations from shared patterns early.
- **Mentorship** - juniors learn from feedback, seniors sharpen by explaining.
- **Quality culture** - when everyone knows code will be read, they write more carefully.

The Optimal Review Size

The SmartBear/Cisco study found:

200-400 lines of code per review

Beyond that, reviewer attention drops sharply.

Keep this in mind: AI-assisted coding can easily produce **1,000+ lines** in a single session.

Your CalcApp morning session likely exceeded the optimal review size.

Traditional vs AI-Assisted Review

Traditional Review: Pain Points

1. **Reviewer fatigue** - attention fades after 200-400 lines
2. **Turnaround time** - waiting for a human can block you for hours or days
3. **Inconsistent depth** - quality depends on who reviews
4. **Style bikeshedding** - humans debate formatting instead of logic

The process works. It has worked for decades.

But it has well-known limits.

What AI Catches Well

- **Null pointer risks** and missing error handling
- **Style inconsistencies** - naming, imports, formatting
- **React hook dependency issues** - subtle and common
- **Simple logic errors** - off-by-one, unreachable code
- **Security anti-patterns** - SQL injection, hardcoded secrets, unescaped input

AI handles **breadth**: scanning every line for patterns it knows.

What AI Consistently Misses

- **Architectural decisions** - "Should this be a separate service?"
- **Business logic correctness** - does the code do what the business needs?
- **Cross-file dependencies** - subtle interactions between distant parts
- **Authorization subtleties** - "edit own profile" vs "edit any profile" look nearly identical in code
- **Whether the code should exist at all** - the most important review question

Humans handle **depth**: intent, architecture, business context.

The Winning Combination

AI handles breadth. Scans every line for known patterns.

Humans handle depth. Understand intent, architecture, context.

Neither alone is sufficient.
Together, they cover the full review spectrum.

Cursor as Code Reviewer

Demo: Flipping the script

Weak vs Effective Prompts

Weak:

```
Review this code.
```

Effective:

```
Review this function for edge cases and error handling.
The input comes from user-submitted form data.
```

```
Check this authentication flow against OWASP top 10.
Flag any missing validation.
```

```
Compare this implementation against the existing pattern
in src/services/userService.ts. Flag inconsistencies.
```

Why Context Changes Everything

The phrase:

> "*The input comes from user-submitted form data*"

completely changes how Cursor evaluates error handling.

Now it knows to look for:

- Input sanitization
- Length limits
- Type coercion issues
- XSS vectors

Specific prompts get specific, actionable results.

5-Step Review Workflow in Cursor

1. **Open the file** you want reviewed
2. **Select the function or section** to focus on
3. **Open Cursor chat** (Ctrl+L) and write a specific prompt
4. **Read the response critically** - not all flags are real problems
5. **Act on valid findings**, dismiss the noise

The critical habit is step 4:

AI review output is input to your judgment, not a checklist to follow blindly.

Categorizing AI Review Findings

Confidence	Type	Action
High	Missing null checks, unused imports, type mismatches	Almost always fix
Medium	Potential race conditions, performance issues	Investigate first
Low	Naming suggestions, structural preferences	Conversation starters

Not every finding is a bug. Your job is to **triage**.

Checkpoint: Your CalcApp Review

Think about the code you built this morning:

- Which parts **handle user input**? (security review candidates)
- Which functions felt **"hacky"** or were rushed? (edge-case analysis)
- Did your style **stay consistent** throughout? (consistency review)

Write down one specific review prompt you would use.

Name the file, the concern, and the context.

Living Documentation

Why static docs rot, and what to do about it

The Documentation Experience

Every developer has had this experience:

1. Onboarding onto a project
2. Find a README or wiki page
3. Within five minutes, realize **it describes a system that no longer exists**

- API endpoints don't match actual routes
- Architecture diagram shows 3 services (there are now 5)
- "Getting Started" fails on step 2

Documentation Rot Is Not a Bug

It is the natural consequence of a fundamental problem:

Code changes continuously.
Documentation is updated manually and sporadically.

The two fall out of sync within weeks, sometimes days.

Misleading documentation is **worse** than no documentation - it creates false confidence.

Documentation Has a Half-Life

Time Since Written	Accuracy
Day 1	~100%
1 month	~80%
6 months	Actively misleading

The decay is inevitable when docs and code are **decoupled**.

What Is Living Documentation?

Documentation that **stays in sync with the codebase** because it is:

- **Generated from the code**, or
- **Tightly coupled to it**

Instead of writing documents *about* code, make the code the **source of truth** and derive documentation from it.

Living Documentation Patterns

- **AI-generated README** from code structure - regenerate when structure changes
- **Docstring generation** - AI documents function signatures, params, return types
- **Changelog automation** - conventional commits (`feat:`, `fix:`) generate changelogs automatically
- **API docs from annotations** - Swagger/OpenAPI extract docs from decorators and types

The docs can't drift because they're **extracted from the source**.

The 80/20 Rule

AI describes WHAT code does.

Humans explain WHY it exists.

The Mechanical 80% (AI Handles)

- Parameter descriptions and types
- Function signatures and return values
- Usage examples derived from tests
- Dependency lists and project structure
- API endpoint documentation

These are **descriptive** - what the code does based on its structure.

The Intentional 20% (Humans Must Write)

- **Why** a particular architecture was chosen over alternatives
- **Context** about business requirements driving technical decisions
- **Warnings** about non-obvious pitfalls from production experience
- **Trade-offs** consciously accepted and the reasoning behind them
- **Historical context** about why something looks "wrong" (but isn't)

Example: The 20% in Action

AI can document:

> `retryCount` *defaults to 3*

Only a human can explain:

> *Changed from 5 to 3 because the downstream service times out after 10 seconds, and 3 retries with exponential backoff fits within that window.*

The 20% is the most valuable part of any documentation.

Key Takeaways

1. **AI makes code review more important, not less** - 1.7x more issues per PR in AI-generated code
2. **AI + human reviewers are complementary** - AI handles breadth, humans handle depth
3. **Specific prompts produce useful reviews** - name the file, the concern, the context
4. **Static docs decay. Living docs stay current** - generate from code, not about code
5. **80/20 rule** - AI writes the what (80%), humans write the why (20%)

AI-Assisted Code Review & Living Documentation

Practice Workshop

Learning Objectives

By the end of this session, you will be able to:

1. **Run an AI-assisted code review** on your CalcApp using Cursor's agent mode
2. **Generate living documentation** - README, inline docs, and component docs
3. **Critically evaluate AI output** - distinguish useful findings from noise

Session Plan

Time	Activity	Duration
0:00	Setup	5 min
0:05	Exercise 1: AI Code Review	25 min
0:30	Exercise 2: Living Documentation	20 min
0:50	Share + Discuss	10 min

Setup

Open your CalcApp in Cursor

Open Your CalcApp Project

1. Open Cursor and load your CalcApp from earlier sessions
2. Verify it runs:

```
cd ~/calcapp
npm install
npm run dev
```

3. **If your CalcApp is broken**, ask the instructor for help.

> *Keep a terminal open alongside Cursor - you will need it to verify fixes.*

Exercise 1

AI Code Review (25 min)

Step 1: Run a Full Project Review (5 min)

Open Cursor's agent mode (`Ctrl+I`) and enter:

```
Review the entire CalcApp project for code quality issues.
Focus on:
- Bug risks and potential runtime errors
- Missing error handling
- Code duplication
- Naming inconsistencies
- TypeScript type safety issues
- Security concerns (e.g., unvalidated user input)

List each issue with file location, severity, and a fix.
```

Read through every finding before acting on anything.

Step 2: Triage the Findings (5 min)

Before applying any fixes, classify each finding:

Category	Meaning
Agree and will fix	Real issue, address now
Agree but defer	Valid point, low priority
Disagree	AI is wrong or overly cautious

Write down your triage (text file, comment, or paper).

The point is not to accept everything - it's to exercise your judgment.

Step 3: Apply Selected Fixes (10 min)

Pick **3-5 issues** from your "agree and will fix" list:

1. Ask Cursor's agent to implement the fix:

```
Fix the [describe issue] in [filename].
Make sure existing functionality is preserved.
```

2. **Review the diff** before accepting - does it introduce new problems?

3. **Verify** nothing broke:

```
npm run dev
# or if you have tests:
npm test
```

Step 4: Probe for Missed Issues (5 min)

General reviews miss deeper problems. Try **targeted prompts**:

```
Are there any edge cases in the budget calculation
logic that could produce incorrect results?
```

```
Check if there are any places where user input flows
into calculations without validation.
```

```
Review the component state management. Are there any
potential race conditions or stale state bugs?
```

Focused prompts catch issues that broad reviews miss.

What to Track for Discussion

Keep notes on these questions:

- What was the **most useful** issue the AI found?
- What obvious problem did the AI **completely miss**?
- What **surprised** you about the review output?
- How many suggestions did you actually **accept**?

Exercise 2

Living Documentation (20 min)

Part A: Generate a README (8 min)

Open Cursor's agent mode and enter:

```
Generate a comprehensive README.md for this CalcApp
project. Include:
- Project description (what it does, who it's for)
- Tech stack
- How to install and run locally
- Project structure overview
- Available scripts (dev, build, test, etc.)
- Key features
```

Review and Improve the README

Check the generated README for:

- **Accuracy** - does it correctly describe CalcApp?
- **Completeness** - is anything important missing?
- **Tone** - is it written for developers who will maintain this code?

Then apply the **80/20 rule**: add 2-3 sentences of human context the AI could not know:

- Why certain architectural decisions were made
- Known limitations or trade-offs
- What the next planned features are

Part B: Generate Inline Documentation (7 min)

Pick your most complex function (budget calculation is a good candidate):

```
Add JSDoc documentation to the functions in [filename].
Include:
- Description of what each function does
- @param descriptions with types
- @returns description
- @throws for any error cases
- @example with a usage example
```

Common AI Documentation Pitfalls

Watch for these in the generated docs:

Pitfall	Example
Restating the obvious	`/** Adds two numbers */ function add(a, b)`
Missing the "why"	Describes *what* the code does, not *why*
Incorrect assumptions	Guesses wrong about edge cases or behavior

Fix at least one doc comment where the AI missed the "why" or got something wrong.

Part C: Generate Component Docs (5 min)

```
Generate documentation for all React components in
this project. For each component, describe:
- Purpose
- Props it accepts (with types)
- Key behaviors
- Usage example
```

Quick scan: would you trust a **new team member** to read this and understand the project?

The 80/20 Documentation Rule

AI handles the 80%: structure, parameter lists, type annotations, install instructions, boilerplate descriptions

You provide the 20%: architectural rationale, business context, known gotchas, "why we did it this way," future plans

Together: comprehensive AND meaningful documentation - in a fraction of the time.

Checkpoint

Before we discuss, verify you have completed:

- Ran an AI code review on your CalcApp
- Triaged the findings (agree / disagree / defer)
- Applied at least 3 fixes from the review
- Generated a README and added human context
- Generated inline docs for at least one function
- Identified at least one thing the AI got wrong

Bonus: Ask Cursor to generate a CHANGELOG from your git history.

Share + Discuss

10 minutes

Discussion Prompts

1. What was the **most useful issue** AI found in your code?
2. What **obvious problem** did the AI miss?
3. How did AI-generated docs compare to **what you would write**?
4. Would you trust AI review to **replace** human review?
5. What **prompting strategies** worked best?

Key Patterns

- AI review is strongest for **pattern-based issues** (naming, style, common bugs) and weakest for **semantic issues** (business logic, architecture)
- AI documentation is strongest for **structural content** and weakest for **rationale and context**
- The value of AI is **speed and coverage**, not replacing human judgment
- **Prompt specificity matters enormously** - targeted questions beat broad prompts

Key Takeaways

1. AI code review is a **powerful first pass**, not a replacement for human review
2. **Targeted prompts beat broad prompts** - specific questions find real bugs
3. **The 80/20 rule** - AI writes the structure, you add the meaning
4. **Always triage before applying** - never accept AI suggestions blindly
5. **Review and documentation are complementary** - both benefit enormously from AI

Introduction to Git

Day 2 - Mastering the Cursor Tool

Learning Objectives

By the end of this session, you will be able to:

1. **Explain why version control is essential** - especially with AI tools
2. **Use core Git commands** - `init`, `add`, `commit`, `log`, `status`, `diff`, `restore`
3. **Apply the Review-Before-Stage pattern** - inspect AI changes before committing
4. **Use checkpoint commits and experiment branches** - protect your work
5. **Set up a Git workflow** for your CalcApp

Why Version Control?

The Save Game Analogy

Imagine playing a long, difficult video game for hours.

Now imagine the game has **no save function**.

Every mistake is permanent. Every bad decision costs you everything.

That's coding without version control.

Why Git Matters More with AI Tools

1. **AI changes are fast and broad** - An agent can modify 10 files in seconds. Without version control, you can't tell what changed.
2. **AI changes need review** - You're the engineer. The AI is the assistant. `git diff` is how you review its work.
3. **Experimentation is cheap** - Branches let you try an AI approach and throw it away if it fails.

Key Concepts

Repository (Repo)

A project folder that Git is tracking.

```
CalcApp/
├── .git/           <-- Git's internal storage (don't touch)
├── src/
├── package.json
└── README.md
```

Run `git init` in a folder and Git creates the hidden `.git/` directory that stores your entire project history.

Commit

A **snapshot** of your entire project at a specific moment.

Like pressing the shutter on a camera - it captures everything exactly as it is.

Each commit has:

- A **unique ID** (hash like `a1b2c3d`)
- A **message** describing what changed and why
- A **pointer** to the previous commit (its parent)

This chain of commits is your project's **timeline**.

Branch

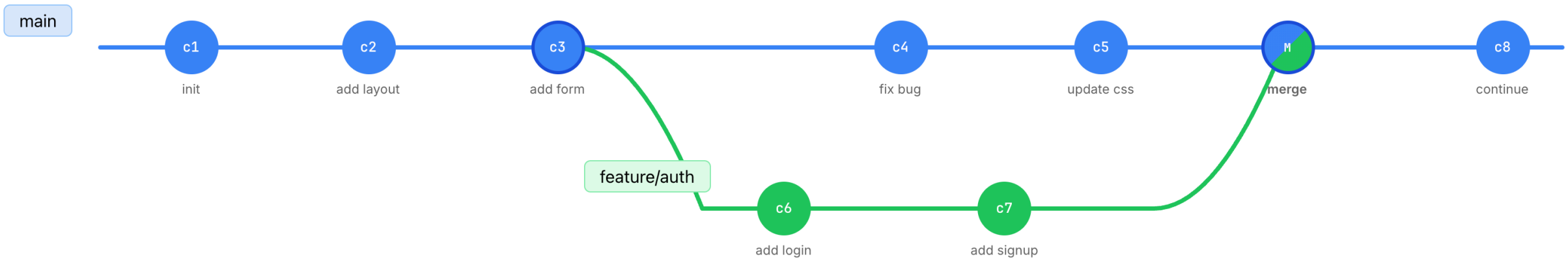
A **parallel timeline**. Create a branch, make changes, and the original timeline stays untouched.

- Like the changes? **Merge** them back.
- Don't like them? **Delete** the branch. Nothing is lost.

This is the "parallel save slots" idea - explore different paths without committing to any of them.

Git: Branches & Merging

Parallel timelines - explore safely, merge when ready



Commit

A snapshot of your project at a specific moment. Each has a unique ID, a message, and a pointer to its parent.

Branch

A parallel timeline. Explore changes without affecting the original. Like them? Merge. Don't? Delete.

Merge

Combine the feature branch back into main. The merge commit has two parents - it joins both timelines.

@ Right-On-Skill

Remote

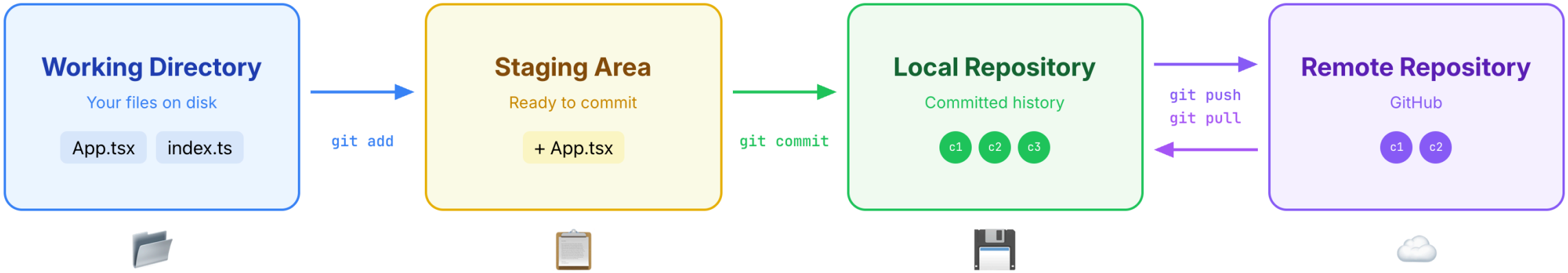
A copy of your repository on a server (like GitHub).

- **Back up** your code
- **Share** it with teammates
- **Collaborate** across machines

The most common remote is called `origin` and points to your GitHub repo.

The Four Areas of Git

How your code travels from editor to GitHub



Photography analogy: 📂 scene → 📋 pose subjects → 💾 click shutter → ☁️ upload to cloud

@ Right-On-Skill

Essential Git Commands

First-Time Setup

Before using Git, configure your identity:

```
# Your name and email (used in commit metadata)
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Set nano as the default editor (friendlier than vim)
git config --global core.editor nano

# Default branch name
git config --global init.defaultBranch main

# Colored output for readability
git config --global color.ui auto
```

Initialize and Check Status

```
# Create a new Git repository
cd ~/CalcApp
git init
```

This creates the `.git/` directory and starts tracking your project.

```
# Check what's changed, staged, and untracked
git status
```

Run `git status` before and after every significant operation. It's your GPS.

Stage Changes

Tell Git which changes to include in your next commit:

```
# Stage a specific file (preferred)
git add src/components/ExpenseForm.tsx

# Stage multiple specific files
git add src/components/ExpenseForm.tsx src/utils/validation.ts
```

Never blindly run `git add .` after an AI tool has made changes.

The AI may have modified files you haven't reviewed yet. Always stage files you've personally inspected.

Review Changes - The Most Important Command

```
# See what changed (unstaged changes)
git diff

# See what's staged and ready to commit
git diff --staged

# See changes in a specific file
git diff src/components/ExpenseForm.tsx
```

`git diff` is your **code review tool for AI output**.

Before you commit anything the AI generated: read the diff. Does the code make sense? Did the AI modify files you didn't expect?

Commit

```
git commit -m "feat(calc): add monthly expense input form"
```

Saves your staged changes as a permanent snapshot.

When working with AI tools, include attribution:

```
git commit -m "feat(calc): add monthly expense input form

Assisted-by: Cursor AI"
```

The message should explain **what** changed and **why**, not how.

View History

```
# See the commit log
git log

# Compact one-line format
git log --oneline

# With a visual branch graph
git log --oneline --graph --all
```

Discard Changes

When an AI suggestion doesn't work out:

```
# Discard changes in a specific file (revert to last commit)
git restore src/components/BrokenComponent.tsx

# Discard ALL uncommitted changes (be careful!)
git restore .

# Unstage a file (keep changes, remove from staging)
git restore --staged src/components/ExpenseForm.tsx
```

Stash Changes

Shelve your current work to try a different approach:

```
# Save current changes to a stash
git stash

# Try a different approach...

# If it doesn't work, get your stashed changes back:
git stash pop
```

Think of it as a temporary pocket for your work-in-progress.

Checkpoint

Initialize Git for your CalcApp

Do This Now

```
# 1. Navigate to your CalcApp directory
cd ~/CalcApp

# 2. Initialize Git
git init

# 3. Check status - see all your untracked files
git status

# 4. Create a .gitignore
echo "node_modules/" > .gitignore

# 5. Stage everything
git add .

# 6. Make your first commit
git commit -m "feat: initial CalcApp structure from Day 2"

# 7. Verify it worked
git log --oneline
```

You should see **one commit** in the log. Your CalcApp now has a safety net.

Git + AI Tools Workflow

Three patterns you will use every day

Pattern 1: Review-Before-Stage

The fundamental workflow. **Every time** an AI tool makes changes:

```
# 1. See what the AI changed
git status

# 2. Review the actual changes, file by file
git diff

# 3. Stage only the files you've reviewed and approved
git add src/components/ExpenseForm.tsx
git add src/utils/validation.ts

# 4. Double-check what you're about to commit
git diff --staged

# 5. Commit with a clear message
git commit -m "feat(calc): add expense validation logic"
```

Never skip step 2. The diff is your code review.

Pattern 2: Checkpoint Commits

"Save game before the boss fight."

```
# Save your current progress
git add .
git commit -m "checkpoint: working state before AI refactor"

# Now let the AI do its thing...

# If it worked: continue and commit the result
# If it broke things: restore your checkpoint
git restore .
```

Use this before:

- Large-scale AI refactors
- Trying a new approach you're unsure about
- AI modifying multiple files at once

Pattern 3: Branch-Per-Experiment

Try an AI suggestion without risking your main codebase:

```
# Create a new branch for the experiment
git checkout -b experiment/ai-auth-feature

# Let the AI work on this branch...
# Review, test, evaluate

# If it worked: merge back to main
git checkout main
git merge experiment/ai-auth-feature

# If it didn't work: delete the branch, nothing lost
git checkout main
git branch -d experiment/ai-auth-feature
```

Multiple experiment branches can run simultaneously.

Collaboration Essentials

Clone, Push, Pull

```
# Clone a project from GitHub
git clone https://github.com/username/CalcApp.git

# Set up the remote and push (first time)
git remote add origin https://github.com/username/CalcApp.git
git push -u origin main

# After that, just push
git push

# Download and integrate changes from the remote
git pull
```

The Daily Collaboration Cycle

```
git pull            <-- Get latest changes from the team
(work on code)      <-- Write code, use AI tools
git status          <-- Review what changed
git diff            <-- Inspect changes
git add <files>     <-- Stage reviewed files
git commit          <-- Save your work
git push            <-- Share with the team
```

You can push your CalcApp to GitHub to preserve progress between sessions.

Best Practices

Conventional Commits

A commit message should answer: "If I read this six months from now, will I understand what changed?"

Format:

```
type(scope): short description
```

Common types:

Type	Meaning
`feat`	A new feature
`fix`	A bug fix
`refactor`	Code restructuring, no behavior change
`docs`	Documentation only
`test`	Adding or fixing tests

Conventional Commits - Examples

```
feat(calc): add monthly budget category selector
```

```
fix(calc): correct expense total calculation rounding error
```

```
refactor(calc): extract validation logic into utility module
```

```
docs(calc): add API documentation for expense endpoints
```

With AI attribution:

```
feat(calc): add monthly expense input form

Assisted-by: Cursor AI
```

Branching Strategy (Keep It Simple)

For this course, use a lightweight strategy:

Branch	Purpose
`main`	Your stable, working code
`feat/<name>`	New features (e.g., `feat/expense-form`)
`experiment/<name>`	AI experiments you might throw away
`fix/<name>`	Bug fixes

Always branch from `main`, and merge back when the feature is working.

Interactive Learning Resource

For visual, hands-on practice with branching and merging:

learngitbranching.js.org

Free interactive tutorial that lets you visualize branch operations in real time. Bookmark it for after the session.

Important Note

> *AI excels at working with Git. If you find Git challenging, or simply want to streamline your workflow, don't hesitate to leverage AI assistance.*

Key Takeaways - Introduction to Git

1. **Git is your safety net** for AI-assisted development - you can't review, revert, or experiment safely without it
2. `git diff` **is the most important command** - your code review tool for AI output
3. **Three AI-specific patterns:** Review-Before-Stage, Checkpoint Commits, Branch-Per-Experiment
4. **Stage selectively, not blindly** - use `git add <file>` for reviewed files, never `git add .` without reviewing
5. **Good commit messages pay off** - use Conventional Commits format, include AI attribution

Questions?

Day 2 - Mastering the Cursor Tool

Tiago Sousa